Estado y efectos en React.

Aprender a utilizar **useState** y **useEffect** en el proyecto existente. Nivel junior · Plan sugerido: 5 sesiones cortas.

HISTORIA DE USUARIO

Como visitante de la tienda, quiero abrir la vista de un producto, cambiar su cantidad y ver cuánto tiempo llevo mirándolo. Al cerrar la vista, sus recursos deben liberarse.

Punto de partida real

El proyecto usa React, Vite y **TypeScript (.tsx)**. Ya existen las rutas, **ProductCard** y los productos de ejemplo. **Shop.tsx** muestra un placeholder. **StrictMode** está habilitado en **src/main.tsx**. Continuá sobre esta base.

Resultado esperado en /shop

Interacción	Comportamiento visible
Abrir producto	Aparece un panel con Sweater ($49), cantidad 1, subtotal $49 y tiempo 0 s.
Botones + y -	La cantidad cambia sin bajar de 1. El subtotal refleja precio × cantidad.
Cambiar cantidad	La pestaña del navegador muestra "Shop | 2 unidades" al elegir 2.
Esperar	El contador avanza aproximadamente una vez por segundo mientras el panel está montado.
Cerrar o navegar a Home	Se desmonta el panel, se detiene su intervalo y se restaura el título previo.
Volver a abrir	Una nueva instancia comienza con cantidad 1 y tiempo 0.

ALCANCE

Un solo producto y estado local. Reutilizá ProductCard. No se requiere API, carrito real, persistencia, store global ni rediseño. Priorizá comprender el comportamiento; no agregues dependencias.

Construí de a un paso.

Primero resolvé las interacciones. Después incorporá cada efecto por separado. La tarea es implementar el ejercicio y poder explicarlo, no copiar una solución completa.

1 / Mostrar y desmontar el panel

- En **src/pages/Shop.tsx**, agregá **isPreviewOpen** con useState(false) y un botón que alterne entre abrir y cerrar.
- Creá **src/components/ProductPreview.tsx** y renderizalo condicionalmente: **{isPreviewOpen && <ProductPreview />}**. Ocultarlo con CSS no lo desmonta.
- Dentro del panel, reutilizá **ProductCard** con name="Sweater" y price={49}. Declaralo como componente fuera de Shop, en su propio archivo.

2 / Cambiar estado y calcular el subtotal

- En ProductPreview, iniciá **quantity** en 1. Agregá botones + y -, y deshabilitá - cuando la cantidad sea 1. Protegé también ese mínimo en la actualización.
- Usá la forma funcional del setter cuando el nuevo valor dependa del anterior: **setQuantity(previous => previous + 1)**. No modifiques el estado directamente.
- Calculá **subtotal = 49 * quantity** durante el render. No necesita otro estado ni un efecto. Los clics se manejan en onClick.

3 / Sincronizar el título con la cantidad

- En ProductPreview, agregá un efecto con **[quantity]**. Dentro, guardá document.title antes de cambiarlo y asigná el título "Shop | N unidades" con la cantidad actual.
- Devolvé una función de limpieza que restaure el título guardado. Observá que se ejecuta antes de repetir el efecto por un cambio de cantidad y también al desmontar.

4 / Iniciar y detener un temporizador

- Agregá **secondsOpen**, inicialmente 0. En otro efecto con **[]**, creá un **window.setInterval** de 1000 ms y conservá su identificador dentro del efecto.
- En cada tick, usá **setSecondsOpen(previous => previous + 1)**. Devolvé una función que llame a **window.clearInterval** con ese mismo identificador.

Entrega acotada: Shop controla la visibilidad; ProductPreview mantiene cantidad, tiempo y sus dos efectos. Conservá las demás rutas y la Home funcionando.

Entender qué está pasando.

Un cambio de estado, un render y un montaje son cosas distintas. Usá el panel para reconocer cada una.

Concepto	En este ejercicio
Estado	Memoria del componente entre renders. Un setter solicita una actualización de la UI; una variable let común no lo hace.
Render	React ejecuta el componente para calcular la UI. Cambiar la cantidad no crea una nueva instancia del panel.
Montaje / desmontaje	Abrir incorpora el panel al árbol; cerrar lo retira. Al volver a montarlo, su estado local empieza nuevamente.
Efecto / cleanup	El efecto sincroniza con algo externo: el título del navegador o su temporizador. La limpieza deshace lo iniciado.

Dependencias: no son un interruptor arbitrario

Sin segundo argumento: el efecto se repite después de cada commit del componente.
[]: no depende de valores reactivos; los renders normales no lo reinician.
[quantity]: corre al montar y cuando cambia quantity. Antes de repetirlo, limpia la ejecución anterior.

STRICTMODE YA ESTÁ ACTIVADO

En desarrollo, React prueba un ciclo extra de setup, cleanup y setup al montar. Podés ver logs repetidos; no quites StrictMode para ocultarlos. La limpieza correcta deja un solo intervalo activo.

Pistas para evitar errores comunes

- Llamá los hooks en el nivel superior del componente, nunca dentro de un if o de onClick. El condicional decide si se renderiza el componente completo.
- El callback del intervalo captura valores de un render. El setter funcional evita depender de un secondsOpen antiguo; por eso el efecto del temporizador no necesita leerlo ni reiniciarse en cada tick.
- No omitas dependencias para forzar un comportamiento. Si agregás otras props o valores reactivos que el efecto utiliza, revisá su lista de dependencias.
- El cleanup debe ser una función: **return () => window.clearInterval(intervalId)**. Ejecutar clearInterval directamente al retornar cancelaría el intervalo de inmediato.

La cantidad y el subtotal son parte de la interacción y del render. El título y el intervalo necesitan sincronización externa. Esa diferencia ayuda a decidir cuándo usar useEffect.

Comprobar y explicar.

Usá **yarn dev** y la consola del navegador. No se requieren tests automáticos para esta práctica.

Checklist de pruebas manuales

- [ ] Entrar en /shop: panel cerrado. Abrirlo: Sweater, cantidad 1, subtotal $49 y contador en 0.
- [ ] Subir a 3: subtotal $147 y título "Shop | 3 unidades". Bajar hasta 1: nunca llega a 0. Cambiar cantidad no reinicia el contador.
- [ ] Agregar temporalmente logs en setup, cleanup y **dentro del callback del intervalo**. Anotar la secuencia observada con StrictMode, sin desactivarlo.
- [ ] Cerrar el panel y esperar unos segundos: no deben aparecer nuevos logs de ticks. El título vuelve al valor anterior. Ver un log de cleanup por sí solo no prueba que el intervalo se haya detenido.
- [ ] Abrir y cerrar tres veces, dejando unos segundos entre acciones: el contador no se acelera ni quedan ticks de instancias anteriores. Cada apertura reinicia ambos estados.
- [ ] Con el panel abierto, ir a Home usando Navbar. Confirmar limpieza y restauración del título. Volver a Shop: panel cerrado; abrirlo crea una instancia nueva.
- [ ] Sin errores en consola, sin intervalos duplicados y sin desbordamiento horizontal en móvil. Retirar los logs de diagnóstico antes de entregar.
- [ ] Ejecutar **yarn lint** y **yarn build**. Registrar los resultados en el PR y corregir los errores introducidos por la tarea.

Preguntas para la revisión

1. ¿Por qué cambiar una variable let no actualiza la pantalla como useState?
2. ¿Qué diferencia hay entre volver a renderizar, montar y desmontar?
3. ¿Cuándo corre el cleanup de [quantity]? ¿Y el del intervalo?
4. ¿Qué puede fallar si olvidás clearInterval o usás un estado capturado antiguo?
5. ¿Por qué el subtotal no requiere useEffect? ¿Qué está comprobando StrictMode?

Plan y entrega

Sesiones 1 a 5: estado y panel; cantidad y subtotal; título y dependencias; intervalo y cleanup; pruebas y PR. Adjuntá una captura o video breve, resultados de las pruebas y respuestas a las preguntas. Mantené los comentarios de código en inglés.

Lecturas oficiales de React

Estado: la memoria de un componente · Referencia de useEffect · Cuándo no hace falta un efecto